

# RoK4 Flat RSL-RL Task 구조 문서

작성일: 2026-07-14  
대상 저장소: /home/rcclab/rok4\_lab  
기준 환경: Isaac Lab v2.3.2, Isaac Sim 5.1.0, env\_isaacclab

## 개요

이번 작업의 목적은 RoK4를 위한 첫 강화학습 환경을 만드는 것이다. 바로 rough terrain으로 가지 않고, 먼저 flat terrain에서 blind velocity walking을 학습할 수 있는 최소 구조를 만들었다.

중요한 점은 G1을 그대로 복사한 것이 아니라는 점이다. G1은 잘 정리된 Isaac Lab humanoid locomotion task 구조를 참고한 템플릿이고, 실제 로봇 asset, 관절 순서, action dimension, reward body 이름은 모두 RoK4 기준으로 다시 작성했다.

현재 생성된 task는 다음 세 개다.

| Task 이름                            | 목적                                                                          |
|------------------------------------|-----------------------------------------------------------------------------|
| RoK4-Isaac-Velocity-Flat-v0        | rok4_train.usd 를 사용하는 RoK4 flat-ground velocity tracking 학습용 task           |
| RoK4-Isaac-Velocity-Flat-Play-v0   | visual mesh가 있는 rok4_test.usd 로 학습된 RoK4 flat policy를 재생하는 task             |
| RoK4-Isaac-Velocity-Flat-Teleop-v0 | rok4_test.usd 와 Isaac Lab SE(2) gamepad/keyboard 입력으로 학습된 policy를 조종하는 task |

## 전체 구조

현재 RoK4 Lab 저장소의 핵심 구조는 다음과 같다.

```
rok4_lab/  
  assets/  
    rok4_wholebody/           # URDF/USD/STL asset bundle, git ignore  
  source/  
    rok4_tasks/  
      rok4_tasks/  
        __init__.py           # RoK4 task registration import  
      assets/  
        robots/  
          rok4.py              # RoK4 asset, actuator, joint order 정의  
      manager_based/  
        locomotion/  
          velocity/  
            config/  
              rok4/  
                __init__.py    # Gym task 등록  
                flat_env_cfg.py # RoK4 Flat env/reward/action/obs 정의  
                contact_force_visualizer.py # 발 접촉력 화살표/숫자 debug view  
                domain_randomization_cfg.py # RoK4 DR 범위와 event mode 정의  
                agents/  
                  rsl_rl_ppo_cfg.py # RSL-RL PPO 설정  
  scripts/  
    check_rok4_zero.py         # 모델/PD zero hold 검사  
    check_rok4_random.py       # sinusoidal command 검사  
    check_rok4_joint_monkey.py # joint axis/limit 검사  
    rsl_rl/  
      _run_isaacclab_rsl.py     # Isaac Lab 원본 train/play wrapper  
      rok4_ppo.py               # KL logging을 추가한 RoK4 PPO/runner  
      train.py                  # RoK4 task/runner 등록 후 Isaac Lab train 실행  
      play.py                   # RoK4 task 등록 후 Isaac Lab play 실행  
      play_teleop.py            # gamepad/keyboard command를 주입하는 teleop 실행
```

## RoK4 구조 관계

현재 RoK4 flat task는 flat\_env\_cfg.py 를 중심으로 움직인다. 다만 실행 entry point는 config/rok4/\_\_init\_\_.py 의 Gym task 등록이고, flat\_env\_cfg.py 는 환경 설정의 main config 역할을 한다.

```
[사용자 실행]  
./isaacclab.sh -p /home/rcclab/rok4_lab/scripts/rsl_rl/train.py  
|  
v
```

```
scripts/rsl_rl/train.py
|
v
scripts/rsl_rl/_run_isaaclab_rsl.py
|   RoK4 package path 추가
|   Isaac Lab 원본 rsl_rl/train.py 실행 전 import rok4_tasks 삽입
|   train에서는 RoK4OnPolicyRunner 삽입
v
source/rok4_tasks/rok4_tasks/__init__.py
|
v
manager_based/locomotion/velocity/config/rok4/__init__.py
|   Gym task 등록
|   id = RoK4-Isaac-Velocity-Flat-v0
+--> env_cfg_entry_point
|     flat_env_cfg.py : RoK4FlatEnvCfg
|
+--> rsl_rl_cfg_entry_point
|     agents/rsl_rl_ppo_cfg.py : RoK4FlatPPORunnerCfg
```

환경 설정 관계는 다음과 같다.

```
RoK4FlatEnvCfg
├─ 상속: IsaacLab LocomotionVelocityRoughEnvCfg
├─ 포함: rewards = RoK4RewardsCfg()
├─ 포함: terminations = RoK4TerminationsCfg()
├─ 학습 사용: ROK4_TRAIN_CFG
├─ 사용: ROK4_JOINT_ORDER
├─ 사용: ROK4_ACTION_SCALE
└─ 호출: apply_rok4_domain_randomization(self)

RoK4FlatEnvCfg_PLAY
├─ 상속: RoK4FlatEnvCfg
└─ 재생 사용: ROK4_TEST_CFG

RoK4RewardsCfg
├─ 상속: IsaacLab RewardsCfg
├─ 부모 reward terms 상속
└─ RoK4 reward terms 정의/override

RoK4TerminationsCfg
├─ 상속: IsaacLab TerminationsCfg
├─ time_out 상속 유지
├─ base_contact 비활성화
└─ illegal_body_contact 추가

RoK4FlatPPORunnerCfg
└─ RSL-RL PPO runner/network/algorithm 설정

RoK4OnPolicyRunner
└─ RoK4PPO 사용
    ├── upstream PPO update/learning-rate 동작 유지
    ├── iteration 평균 KL -> Loss/kl
    └─ iteration 최대 KL -> Loss/kl_max

RewardTermCfg(func=mdp.xxx)
├─ RoK4 로컬 mdp reward 함수 호출
├─ Isaac Lab 공통 mdp reward 함수 호출
└─ locomotion velocity 전용 mdp reward 함수 호출
```

각 파일의 관계를 역할로 나누면 다음과 같다.

| 파일                          | 관계              | 역할                                                                                                                                                        |
|-----------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| config/rok4/__init__.py     | task 등록         | Train, Play, Teleop task 이름을 각각의 RoK4 env cfg 와 RoK4FlatPPORunnerCfg 에 연결                                                                                 |
| flat_env_cfg.py             | 환경 main config  | RoK4 scene, terrain, action, observation, reward, command, termination 설정                                                                                 |
| domain_randomization_cfg.py | DR config       | RoK4 foot material, base mass, COM, external wrench, reset randomization 범위와 event mode 설정                                                                |
| mdp/__init__.py             | mdp re-export   | Isaac Lab locomotion mdp를 다시 export하고 RoK4 로컬 mdp 함수를 함께 노출                                                                                               |
| mdp/rewards.py              | RoK4 reward 계산식 | RoK4 전용 action_pos_limits 및 weighted joint_torques_l2, joint_vel_l2, joint_acc_l2, action_rate_l2, second_action_rate_l2 등의 reward raw value 계산 함수/클래스 정의 |
| agents/rsl_rl_ppo_cfg.py    | 학습 config       | RSL-RL PPO network와 algorithm hyperparameter 설정                                                                                                           |

| 파일                           | 관계                 | 역할                                                                                |
|------------------------------|--------------------|-----------------------------------------------------------------------------------|
| scripts/rsl_rl/rok4_ppo.py   | 학습 algorithm 확장    | adaptive learning rate가 계산하는 mini-batch KL의 iteration 평균/최대값을 TensorBoard에 기록     |
| assets/robots/rok4.py        | robot asset config | USD path, initial pose, actuator, joint order, action scale, self-collision 설정 제공 |
| IsaacLab velocity_env_cfg.py | 부모 config          | manager-based velocity locomotion의 공통 scene/action/obs/reward/termination 를 제공    |
| IsaacLab mdp/rewards.py      | reward 함수 모음       | Isaac Lab 공통 reward 함수와 locomotion velocity 전용 reward 함수를 mdp.xxx namespace로 제공   |

한 줄로 요약하면, flat\_env\_cfg.py 는 RoK4 환경의 중심 설정 파일이고, \_\_init\_\_.py 는 task 이름을 등록하는 입구, rsl\_rl\_ppo\_cfg.py 는 학습기 설정, rok4.py 는 로봇 asset 설정이다.

## 새로 생성된 파일

### source/rok4\_tasks/rok4\_tasks/manager\_based/\_\_init\_\_.py

Manager-based task 패키지 시작점이다. Isaac Lab의 ManagerBasedRLEnv 스타일 task를 RoK4 저장소 안에 넣기 위한 디렉터리 계층이다.

### source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/\_\_init\_\_.py

Locomotion task 그룹 시작점이다. 향후 velocity walking 외에 balancing, standing, whole-body tracking 같은 locomotion 계열 task를 추가할 때 같은 계층 아래에 둘 수 있다.

### source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/\_\_init\_\_.py

Velocity tracking task 그룹 시작점이다. 현재 RoK4의 첫 학습 목표가 command velocity를 따라 걷는 것이므로 이 계층을 만들었다.

### source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/config/\_\_init\_\_.py

Velocity task 안에서 robot별 config를 묶기 위한 계층이다. 지금은 rok4 만 있지만, 나중에 RoK4 variant가 늘어나면 같은 구조 아래에 추가할 수 있다.

### source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/config/rok4/\_\_init\_\_.py

Gymnasium task를 등록하는 파일이다. 여기에서 다음 task 이름이 Isaac Lab registry에 올라간다.

```
RoK4-Isaac-Velocity-Flat-v0
RoK4-Isaac-Velocity-Flat-Play-v0
RoK4-Isaac-Velocity-Flat-Teleop-v0
```

각 task는 isaacsim.envs:ManagerBasedRLEnv 를 entry point로 사용한다. 환경 설정은 flat\_env\_cfg.py 에서, RSL-RL PPO 설정은 agents/rsl\_rl\_ppo\_cfg.py 에서 불러온다.

### source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/config/rok4/flat\_env\_cfg.py

이번 작업의 핵심 파일이다. RoK4 flat walking 환경을 정의한다.

주요 역할은 다음과 같다.

| 항목                   | 내용                                                                                                                            |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Scene                | 학습 task는 ROK4_TRAIN_CFG 를 {ENV_REGEX_NS}/Robot 위치에 spawn                                                                      |
| Terrain              | plane terrain 사용, rough terrain generator와 height scanner 제거                                                                  |
| Action               | ROK4_JOINT_ORDER 기준 13차원 joint position target                                                                                |
| Observation          | blind proprioceptive history observation                                                                                      |
| Domain randomization | domain_randomization_cfg.py 의 apply_rok4_domain_randomization(self) 호출                                                        |
| Reward               | velocity tracking, upright, action smoothness, 전체 13관절의 실제/목표 soft position limit, torque/acc penalty, foot contact 관련 reward |
| Termination          | 부모 timeout 유지, Foot를 제외한 모든 body의 illegal contact                                                                             |
| Play cfg             | ROK4_TEST_CFG 로 교체, 적은 env 수, corruption off, push/random external force off                                                  |

| 항목         | 내용                                                                             |
|------------|--------------------------------------------------------------------------------|
| Teleop cfg | Play cfg를 상속하고 1개 env, 600초 timeout, heading off, 자동 command resampling off 설정 |

RoK4 action space는 다음 13축이다.

```

Left leg:
  L_Hip_Yaw_Joint
  L_Hip_Roll_Joint
  L_Hip_Pitch_Joint
  L_Knee_Pitch_Joint
  L_Ankle_Pitch_Joint
  L_Ankle_Roll_Joint

Right leg:
  R_Hip_Yaw_Joint
  R_Hip_Roll_Joint
  R_Hip_Pitch_Joint
  R_Knee_Pitch_Joint
  R_Ankle_Pitch_Joint
  R_Ankle_Roll_Joint

Torso:
  Torso_Yaw_Joint

```

관절 순서는 반드시 rok4.py 의 ROK4\_JOINT\_ORDER 를 따른다. G1의 관절 순서를 그대로 쓰지 않는다.

Action pipeline과 clip

RoK4 policy가 출력하는 action은 먼저 RSL-RL wrapper에서 clip\_actions = 1.0 설정을 통해 [-1, 1] 로 잘린다. 그 다음 RoK4 action scale과 default pose offset이 적용되어 실제 joint position target이 된다.

```

actor network output
-> RslRLVecEnvWrapper clip_actions = 1.0
-> clipped_raw_action in [-1, 1]
-> q_target = default_joint_pos + clipped_raw_action * ROK4_ACTION_SCALE
-> IdealPDActuatorCfg explicit torque-PD

```

중요한 점은 last\_action observation이 q\_target 이나 clipped\_raw\_action \* ROK4\_ACTION\_SCALE 이 아니라 clipped raw policy action이라는 점이다. 즉 observation 안의 action 항목은 "이전 step에서 policy가 낸 action"을 의미하고, motor target은 별도로 action scale과 default pose를 거쳐 만들어진다.

반면 action\_rate\_l2 와 second\_action\_rate\_l2 reward는 raw action 차이를 그대로 쓰지 않고, clipped\_raw\_action \* ROK4\_ACTION\_SCALE 차이를 사용한다. 즉 observation은 raw action 의미를 유지하고, smoothness reward는 실제 joint target offset 변화량에 더 가까운 값으로 계산한다.

ROK4\_ACTION\_SCALE 은 기존 Isaac Gym RoK4의 actuator action scale과 동일하다. 좌측 다리, 우측 다리, torso 순서의 값은 다음과 같다.

```

[0.4, 0.5, 1.25, 1.5, 0.75, 0.75,
0.4, 0.5, 1.25, 1.5, 0.75, 0.75,
0.4]

```

이 하나의 scale 배열을 joint target 계산과 두 action smoothness reward가 공통으로 사용한다.

학습 command 범위

Flat 학습은 lin\_vel\_x=(-0.1, 0.85) m/s, lin\_vel\_y=(-0.3, 0.3) m/s, ang\_vel\_z=(-0.6, 0.6) rad/s 를 사용한다. x command의 제한된 음수 영역은 후진 보행 curriculum의 중간 단계이며, 이후 기존 Isaac Gym RoK4 범위인 (-0.3, 0.85) m/s 까지 확장할 수 있다.

feet\_air\_time\_positive\_biped 는 threshold=0.55 s, weight=0.75 를 사용한다. threshold는 정확한 gait period 목표가 아니라 보상되는 single-stance 시간의 상한이다. weight를 유지한 채 threshold를 높였으므로 최대 pre-dt 항목 크기는 0.30 에서 0.4125 로 증가한다.

Play 환경은 lin\_vel\_x=0.85 m/s 의 고정 전진 명령을 사용하고 lin\_vel\_y=0.0 m/s, ang\_vel\_z=0.0 rad/s 로 고정한다. Teleop 환경은 자동 표본화를 끄고 사용자가 입력한 base-frame [lin\_vel\_x, lin\_vel\_y, ang\_vel\_z] 를 command buffer에 직접 기록한다.

초기 root 위치

기존 Gym 메모에서 gait-ready CoM 기준 (0.0575 / 2, 0, 0.835) m 는 gait-ready base 기준 (0.0552, 0, 0.907) m 에 대응한다. 다리를 펴고 선 자세의 base 높이는 z=0.919 m 이다. ROK4\_TRAIN\_CFG 의 실제 articulation root 초기 위치는 (0.0552, 0.0, 0.929) m 이며, 이는 다리를 편 직립 base 높이보다 0.010 m 위에 로봇을 살짝 띄워 배치한 값이다.

이 pos 는 root의 world/environment 위치이고, 초기 관절 자세는 \_ROK4\_INIT\_JOINT\_POS 가 별도로 정의한다. 현재 \_ROK4\_INIT\_JOINT\_POS 도 Gym의 활성 defaultJointAngles 및 normalJointAngles gait-ready 자세와 동일하게 맞춰져 있다.

|                   |             |
|-------------------|-------------|
| Gym과 동일한 초기 관절 자세 |             |
| 관절 그룹             | 초기 각도 [rad] |

| 관절 그룹                  | 초기 각도 [rad] |
|------------------------|-------------|
| Hip Yaw / Hip Roll     | 0.0         |
| Hip Pitch              | -0.0924     |
| Knee Pitch             | 0.345       |
| Ankle Pitch            | -0.253      |
| Ankle Roll / Torso Yaw | 0.0         |

use\_default\_offset=True 이므로 이 초기 관절 자세는 policy action에 더해지는 joint position target의 기준 자세로도 사용된다.

Isaac Lab의 train.py 와 play.py 를 사용할 때는 RslRlVecEnvWrapper 가 자동으로 clipping을 수행한다. 반면 ONNX/TorchScript policy를 직접 호출하는 sim2sim 또는 sim2real 코드에서는 같은 동작을 외부 코드에서 직접 수행해야 한다.

```

policy_output = policy(obs)
clipped_raw_action = torch.clamp(policy_output, -1.0, 1.0)
q_target = default_joint_pos + clipped_raw_action * ROK4_ACTION_SCALE
last_action = clipped_raw_action

```

Actor observation은 5-step history를 사용한다.

```

base_ang_vel
projected_gravity
velocity_commands
joint_pos
joint_vel
last_action

```

Actor에는 다음 정보를 넣지 않는다.

```

camera image
terrain height scan
base linear velocity

```

따라서 현재 구조는 estimator가 없는 history MLP 방식의 blind walking baseline이다.

source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/config/rok4/domain\_randomization

RoK4 flat task의 domain randomization 값을 따로 관리하는 파일이다. flat\_env\_cfg.py 안에 긴 DR block을 직접 두지 않고, 다음 한 줄로 적용한다.

```

apply_rok4_domain_randomization(self)

```

이 분리의 목적은 reward, observation, action 설정과 DR 튜닝값을 섞지 않는 것이다. 앞으로 friction, restitution, mass, COM, reset perturbation을 조정할 때 이 파일을 먼저 보면 된다.

현재 DR 구조는 다음과 같다.

| 그룹                       | mode    | 현재 설정                                                                                     |
|--------------------------|---------|-------------------------------------------------------------------------------------------|
| Foot physics material    | startup | L_Foot_Link, R_Foot_Link 의 static friction 0.8, dynamic friction 0.6, restitution 0.1~0.3 |
| Body mass                | startup | base, upper, lower body group을 각각 0.9~1.25 배로 scale                                       |
| Body COM                 | startup | base, upper, lower body group의 COM offset을 그룹 별 범위로 randomize                             |
| External base wrench     | reset   | 현재 force/torque range는 0으로 두어 외력은 꺼진 상태                                                   |
| Reset joint pose         | reset   | default joint position을 0.9~1.1 scale                                                     |
| Reset base pose/velocity | reset   | episode reset마다 base x/y/yaw pose와 base velocity를 약하게 randomize                           |

startup DR은 scene 생성 시 각 environment에 대해 샘플링된다. reset DR은 episode reset마다 다시 샘플링된다. 4096개 environment 학습에서 CPU-heavy DR을 매 reset 수행하지 않기 위해 material, mass, COM은 현재 startup 으로 둔다.

Mass와 COM DR은 다음 세 body group으로 나누어 관리한다.

| 그룹    | 링크                                | mass scale | COM range       |
|-------|-----------------------------------|------------|-----------------|
| base  | Base_Link                         | 0.9~1.1    | x/y/z +-0.01 m  |
| upper | Upper_Body_Link                   | 0.9~1.25   | x/y/z +-0.03 m  |
| lower | left/right leg 전체 link와 foot link | 0.9~1.25   | x/y/z +-0.005 m |

RoK4 control timing

RoK4는 Isaac Lab 본체의 LocomotionVelocityRoughEnvCfg 를 수정하지 않고, RoK4FlatEnvCfg 안에서 simulation/control timing만 override한다.

| 설정                           | 값                 |
|------------------------------|-------------------|
| sim.dt                       | 0.002 s           |
| physics frequency            | 500 Hz            |
| decimation                   | 5                 |
| policy/action period         | 0.010 s           |
| policy/action frequency      | 100 Hz            |
| contact sensor update period | 0.002 s           |
| contact-force history length | 5 physics samples |

Contact sensor의 history\_length 는 Digit locomotion 설정과 같은 방식으로 self.decimation 에 맞춘다. 따라서 현재는 10 ms policy interval 동안 실행되는 5개의 2 ms physics step마다 contact-force sample 하나를 보존한다. 이 이력은 contact 기반 reward와 termination에서 짧은 접촉을 확인하기 위한 sensor buffer이며, actor 입력을 5 frame 쌓는 self.observations.policy.history\_length 와는 서로 다른 설정이다.

source/rok4\_tasks/rok4\_tasks/manager\_based/locomotion/velocity/config/rok4/agents/rsl\_rl\_ppo\_c

RSL-RL PPO runner 설정 파일이다.

현재 설정은 RoK4용 network/observation normalization 설정에 G1-style PPO algorithm 값을 섞은 flat walking 초기 baseline이다. 이 값들은 최종 튜닝값이 아니라 첫 RoK4 flat 보행 실험을 위한 시작점이다.

| 설정                             | 값                                      |
|--------------------------------|----------------------------------------|
| experiment name                | rok4_flat                              |
| max iterations                 | 5000                                   |
| steps per env                  | 24                                     |
| actor hidden dims              | [512, 256, 128]                        |
| critic hidden dims             | [512, 256, 128]                        |
| actor/critic obs normalization | True                                   |
| action clipping                | clip_actions = 1.0                     |
| activation                     | elu                                    |
| learning rate                  | 1.0e-3                                 |
| entropy coef                   | 0.002                                  |
| value loss coef                | 1.0                                    |
| desired KL                     | 0.01                                   |
| obs groups                     | actor와 critic 모두 policy observation 사용 |

첫 버전에서는 asymmetric critic이나 estimator를 넣지 않았다. Flat walking이 먼저 돌아가는지 확인한 뒤, rough terrain이나 estimator를 단계적으로 추가하기 위함이다.

entropy\_coef 는 exploration standard deviation을 키우는 방향의 entropy 항에 곱해지는 계수다. 현재 0.002 는 최초 0.008 과 저-noise 실험값 0.001 사이의 중간 설정이다. 최초 설정보다 특정 관절의 noise standard deviation이 과도하게 커지는 현상을 줄이면서, 0.001 에서 관찰된 강한 exploration 감소를 완화하기 위한 값이다.

desired\_kl=0.01 은 old policy와 update 중인 new policy의 Gaussian action distribution 차이를 관리하는 adaptive learning-rate 기준이다. 현재 RSL-RL은 각 mini-batch KL이 0.02 보다 크면 learning rate를 1.5 로 나누고,  $0 < KL < 0.005$  이면 1.5 배하며, 그 사이에서는 유지한다.

scripts/rsl\_rl/rok4\_ppo.py

Isaac Lab 또는 설치된 rsl\_rl 파일을 수정하지 않고 KL을 TensorBoard에 기록하기 위한 RoK4 로컬 PPO/runner 확장이다. RoK4PP0 는 upstream PPO의 adaptive KL 계산값을 누적하고, RoK40nPolicyRunner 는 학습 시 이 PPO를 생성한다.

현재 num\_learning\_epochs=5, num\_mini\_batches=4 이므로 한 PPO iteration에는 20개의 mini-batch KL이 계산된다.

| KL TensorBoard tags |                                         |
|---------------------|-----------------------------------------|
| tag                 | 의미                                      |
| Loss/kl             | 한 iteration의 20개 mini-batch KL 평균       |
| Loss/kl_max         | 같은 iteration에서 관측된 최대 mini-batch KL     |
| Loss/learning_rate  | adaptive KL schedule 적용 후 learning rate |

Loss/kl 은 PPO의 Loss/surrogate 와 다른 값이다. Surrogate는 actor 최적화 목적함수이고, KL은 old/new policy가 얼마나 달라졌는지를 측정한다. 기존 TensorBoard event 파일에는 새 tag가 소급 추가되지 않으며, 수정 후 RoK4 train.py 로 시작한 run부터 기록된다.

scripts/rsl\_rl/\_run\_isaaclab\_rsl.py

Isaac Lab 원본 scripts/reinforcement\_learning/rsl\_rl/train.py 와 play.py 를 수정하지 않고 RoK4 task와 선택적인 teleop 입력을 삽입하기 위한 wrapper이다.

동작 흐름은 다음과 같다.

1. rok4\_lab/source/rok4\_tasks 를 sys.path에 추가
2. 현재 작업 디렉터리의 Isaac Lab RSL-RL script 경로 찾기
3. Isaac Lab 원본 train.py/play.py source를 읽기
4. 원본의 import isaacsim\_tasks 바로 뒤에 import rok4\_tasks 삽입
5. train.py에는 RoK4OnPolicyRunner import/생성을 삽입
6. teleop이면 device CLI, SE(2) 입력 생성, command update를 play.py에 삽입
7. 원본 script를 실행

이 방식의 장점은 Isaac Lab 본체를 수정하지 않는다는 점이다.

## scripts/rsl\_rl/train.py

RoK4 task와 RoK4OnPolicyRunner 를 등록한 뒤 Isaac Lab 원본 RSL-RL training script를 실행한다.

사용자는 Isaac Lab root에서 다음처럼 실행한다.

```
./isaacsim.sh -p /home/rclab/rok4_lab/scripts/rsl_rl/train.py \  
--task RoK4-Isaac-Velocity-Flat-v0 \  
--num_envs 512 \  
--max_iterations 5000 \  
--headless
```

## scripts/rsl\_rl/play.py

RoK4 task를 등록한 뒤 Isaac Lab 원본 RSL-RL playback script를 실행한다.

사용자는 Isaac Lab root에서 다음처럼 실행한다.

```
./isaacsim.sh -p /home/rclab/rok4_lab/scripts/rsl_rl/play.py \  
--task RoK4-Isaac-Velocity-Flat-Play-v0 \  
--num_envs 16 \  
--checkpoint /path/to/model.pt
```

## Contact force debug visualization

contact\_force\_visualizer.py 의 RoK4ContactForceVisualizer 는 Isaac Lab의 기존 ContactSensor 를 상속한다. 보상과 termination은 기존 contact\_forces sensor data를 그대로 사용하고, debug visualization을 켜를 때만 환경 0의 좌우 발 데이터를 읽어 다음 요소를 표시한다.

- 왼발: 파란색 world-frame 전체 지면반력 화살표
- 오른발: 초록색 world-frame 전체 지면반력 화살표
- RoK4 Contact Forces 창: 좌우 발의  $|F|$  를 newton 단위 숫자로 표시

Isaac Sim UI의 Scene Debug Visualization 에서 Contact Forces 를 체크하면 화살표와 숫자 창이 함께 켜지고, 체크를 해제하면 함께 숨겨진다. Flat task의 지면 collision prim을 filter로 지정하고 PhysX가 별도로 제공하는 world-frame 법선 접촉력과 접선 접촉력을 발별로 더한다.

$$\mathbf{F}_{GRF}^w = \mathbf{F}_{normal}^w + \mathbf{F}_{tangential}^w = [F_x^w, F_y^w, F_z^w]$$

발마다 이 합력 벡터 하나를 표시한다. 화살표 방향은  $\mathbf{F}_{GRF} / |\mathbf{F}_{GRF}|$  이고, 화살표 길이와 숫자는  $|\mathbf{F}_{GRF}| = \sqrt{F_x^2 + F_y^2 + F_z^2}$  에 비례한다. X/Y/Z 축마다 별도 화살표를 만드는 구조가 아니다. 기본 arrow\_x.usd 의 local x 범위가 [-0.25, 0.75] 이므로 화살표 원점을 합력 방향으로 보정하여 꼬리가 지면 아래로 들어가지 않고 발바닥 바로 위에서 시작하도록 한다. 가독성을 위해 화살표 길이에 display-only 상한을 적용한다.

이 값은 발과 지면 사이의 전체 접촉 합력이며, 발목에 설치한 6축 F/T sensor의  $[F_x, F_y, F_z, M_x, M_y, M_z]$  출력은 아니다. 4096-env 학습에서 불필요한 GPU-to-CPU/UI 비용이 발생하지 않도록 debug view 기본값은 off이며, 켜를 때도 환경 0만 시각화한다. 구현과 설정은 모두 rok4\_lab 안에 있고 Isaac Lab 원본은 수정하지 않는다.

## scripts/rsl\_rl/play\_teleop.py

기존 Play task는 변경하지 않고, Teleop task에서만 학습된 policy의 velocity command를 수동 입력으로 교체한다. Se2Gamepad 는 Omniverse gamepad interface를 직접 사용하므로 ROS 2 joy\_node, Isaac Sim ROS 2 Bridge, /joy subscriber 또는 별도 IPC가 필요하지 않다.

Gamepad 실행:

```
./isaacsim.sh -p /home/rclab/rok4_lab/scripts/rsl_rl/play_teleop.py \  
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \  
--teleop_device gamepad \  
--teleop_dead_zone 0.05 \  
--checkpoint /path/to/model.pt \  
--real-time
```

Gamepad의 left stick 위/아래는 x 선속도를 명령한다. Left stick 오른쪽은 음의 y 선속도, 왼쪽은 양의 y 선속도이며, right stick 오른쪽은 음의 yaw, 왼쪽은 양의 yaw를 명령하므로 스틱 방향과 로봇의 이동/회전 방향이 일치한다. 정규화된 입력은 [-1, 1] 로 clip한 뒤 학습 범위  $v_x=(-0.1, 0.85)$  m/s,  $v_y=(-0.3, 0.3)$  m/s,  $w_z=(-0.6,$

0.6) rad/s 로 scale한다.

Keyboard 실행:

```
./isaacsim.sh -p /home/rclab/rok4_lab/scripts/rsl_rl/play_teleop.py \
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \
--teleop_device keyboard \
--checkpoint /path/to/model.pt \
--real-time
```

Keyboard는 Up/Down으로 전진/후진, Left/Right로 횡이동, Z/X 로 양/음 yaw, L 로 누적 command를 초기화한다. 향후 keyboard mapping을 바꾸더라도 --teleop\_device keyboard entry point는 유지한다.

Teleop command는 100 Hz play loop 시작 시 command buffer에 기록된다. 다만 그 시점의 policy observation은 직전 environment step에서 이미 생성되어 있으므로 현재 action에는 직전 command observation이 사용된다. 새 command는 이어지는 step 후 observation을 통해 다음 policy loop에서 보이며, 수동 입력부터 policy 반영까지 최대 한 policy period인 약 10 ms 가 걸린다.

## 수정된 기존 파일

### source/rok4\_tasks/rok4\_tasks/\_\_init\_\_.py

기존에는 패키지 docstring만 있었다. 이제 RoK4 task registry가 import되도록 다음 역할을 한다.

```
from .manager_based.locomotion.velocity.config import rok4
```

이 import가 실행되어야 RoK4-Isaac-Velocity-Flat-v0 task가 gym registry에 등록된다.

### README.md

새 Flat RL Task 섹션을 추가했다. task 이름, observation/action 구조, smoke test, normal training, play 명령어, DR 관리 파일, self-collision 설정을 문서화했다. Teleop task, gamepad/keyboard 입력, command scale과 한 policy-step 입력 지연도 함께 문서화했다. Contact Forces debug toggle로 환경 0의 좌우 발 접촉력 화살표와 실시간 newton 값을 확인하는 방법도 문서화했다.

### CHANGELOG.md

Unreleased 항목에 RoK4 flat RSL-RL task 추가 내용을 기록했다.

## 학습 실행 흐름

학습 명령을 실행하면 전체 흐름은 다음과 같다.

```
./isaacsim.sh
-> /home/rclab/rok4_lab/scripts/rsl_rl/train.py
-> _run_isaacsim_rsl.py
-> sys.path에 rok4_tasks 추가
-> Isaac Lab 원본 train.py 실행
-> import isaacsim_tasks
-> import rok4_tasks
-> RoK4 task gym 등록
-> hydra_task_config가 env_cfg / agent_cfg 로드
-> gym.make("RoK4-Isaac-Velocity-Flat-v0")
-> ManagerBasedRLEnv 생성
-> RslRlVecEnvWrapper
-> RoK4OnPolicyRunner
-> RoK4PP0 학습
-> Loss/kl, Loss/kl_max 기록
```

## 스모크 테스트 결과

다음 smoke test를 수행했다.

```
./isaacsim.sh -p /home/rclab/rok4_lab/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 2 \
--max_iterations 1 \
--headless
```

확인된 결과는 다음과 같다.

| 항목    | 결과 |
|-------|----|
| 환경 생성 | 성공 |



| 항목                | 결과                                                                        |
|-------------------|---------------------------------------------------------------------------|
| action shape      | 13                                                                        |
| observation shape | 240                                                                       |
| actor input       | 5-step history 기반 240차원                                                   |
| actor output      | 13차원 joint position target                                                |
| PPO 1 iteration   | 성공                                                                        |
| 생성 checkpoint     | /home/rclab/IsaacLab/logs/rsl_rl/rok4_flat/2026-07-07_13-44-26/model_0.pt |

## 현재 설계 의도

현재 task는 최종 rough terrain 정책이 아니다. 첫 번째 목표는 원인 분리가 쉬운 flat walking baseline이다.

현재 단계에서 의도적으로 넣지 않은 항목은 다음과 같다.

```
rough terrain
terrain curriculum
height scanner
camera observation
explicit estimator / MLP encoder
teacher-student distillation
asymmetric privileged critic
```

추후 권장 순서는 다음과 같다.

- 1. Flat smoke test 확인
- 2. Flat 학습으로 서기/걸기 안정화
- 3. reward, action scale, termination 조정
- 4. RoK4 Rough task 추가
- 5. weak rough terrain curriculum
- 6. estimator 또는 teacher-student 구조 추가

## 주요 실행 명령어

Smoke test:

```
cd /home/rclab/rok4_lab
ROK4LAB_DIR=$(pwd)

cd /home/rclab/IsaacLab
conda activate env_isaacclab

./isaacclab.sh -p ${ROK4LAB_DIR}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 2 \
--max_iterations 1 \
--headless
```

Normal training:

```
./isaacclab.sh -p ${ROK4LAB_DIR}/scripts/rsl_rl/train.py \
--task RoK4-Isaac-Velocity-Flat-v0 \
--num_envs 512 \
--max_iterations 5000 \
--headless
```

Play:

```
./isaacclab.sh -p ${ROK4LAB_DIR}/scripts/rsl_rl/play.py \
--task RoK4-Isaac-Velocity-Flat-Play-v0 \
--num_envs 16 \
--checkpoint /path/to/model.pt
```

Teleop with gamepad:

```
./isaacclab.sh -p ${ROK4LAB_DIR}/scripts/rsl_rl/play_teleop.py \
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \
--teleop_device gamepad \
--teleop_dead_zone 0.05 \
--checkpoint /path/to/model.pt \
--real-time
```

Teleop with keyboard:

```
./isaacclab.sh -p ${ROK4LAB_DIR}/scripts/rsl_rl/play_teleop.py \
--task RoK4-Isaac-Velocity-Flat-Teleop-v0 \
--teleop_device keyboard \
--checkpoint /path/to/model.pt \
--real-time
```